DDMI Logic Conversion

LabVIEW **800G_DDMI_Cal V6.8** → Python `ddmi-cal-python` (branch `engine`) · AWS 800G OSFP 2×FR4
One section per test step: what V6.8 does, in which order, with which registers, waits, maths and limits, and how the Python port does the same step.

> **How to read each section.** Every section uses the same layout:
>
> 1. **Card**: purpose, V6.8 VI, Python function, config block, result columns.
> 2. **Steps**: one table row per action, with the register (page:byte), value and wait.
> 3. **Calculations** and **Limits / pass-fail / retries**.
> 4. **V6.8 vs Python** (purple box): every place where the Python code is not a 1:1 copy, and why.
>
> Tags: HW confirmed on a real module · OPEN not fully decoded · PY Python-only addition. All values were read from the V6.8 block diagrams (constants, case names, error-wire order).

Contents

0	Whole run: order, loops and time budget
1	Start-up: Reini, Set_Temp, Read SN, VCC, Read_Spec, Wait_Temp
2	**MPD_Scan**: sweep, Peak Search, operating points, TOPS write
3	Adjust Power: bias set-points
4	Scan_Disable: TX-disable (tops_off) tuning
5	**TX Cal**: DDMI_Adjust_TX
6	**RX Cal**: DDMI_RXP
7	**VCC Cal**: DDMI_VCC
8	Temperature: DDMI_Temp
9	ShowPF: final module clean-up and PASS/FAIL
10	MES, records and data storage
11	Station calibrations: Tx Loss, RX attenuation tables
12	Python-only safety additions
13	All delays in one table
14	Register master list
15	Units and formulas
16	Open items and differences still to close

0. Whole run: order, loops and time budget

```
MES (Cimes_Interface.vi)  Pcs_Check_IN -> Read_Date -> Read_Spec -> Read_Test_Times
      |
1  Start-up       Reini -> Set_Temp -> Read SN -> Read_Spec -> Wait_Temp (skipped) -> VCC
      |
   StartTest queue: Scan_MPD -> DDMI_CAL -> ShowPF -> Move_out -> Stop
      |
   Scan_MPD  outer loop x2 (stop when Scan_Disable passes)
2     |- MPD Monitor   inner loop x2 (stop when the scan passes)
3     |- Adjust Power  only on outer pass 1, only if Only_Test = False
4     '- Scan_Disable
   DDMI_CAL  each item: Cal -> Check, re-run once (after 50 ms) if the check fails
5     |- DDMI_Adjust_TX   (TX Cal)
6     |- DDMI_RXP         (RX Cal)
7     |- DDMI_VCC         (VCC Cal)
8     '- DDMI_Temp
9  ShowPF    temp-offset off, exit test mode, reset, DSP-mode check, PASS/FAIL
10 Move_out  Save_Data (3 tries) -> Pcs_Check_OUT -> local Record CSV
```

V6.8 state	Python (ddmi/flow.py)	UI stage light
Reini, Set_Temp, Read SN, Wait_Temp, VCC	`startup.reini / set_temp / read_sn / wait_temp / set_vcc`	Startup
Scan_MPD (outer loop of 2)	`flow.run` loop over `flow.scan_mpd_passes` = 2	Scan MPD
MPD Monitor (inner loop of 2)	`mpd_scan.scan_mpd` (`scan_attempts` = 2)	Scan MPD
Adjust Power (pass 1 only)	`mpd_scan.adjust_power` (attempt == 1)	Scan MPD
Scan_Disable	`mpd_scan.scan_disable`	Scan MPD
DDMI_Adjust_TX	`tx_cal.tx_cal` + `tx_cal.tx_check`	TX Cal
DDMI_RXP	`rx_cal.rx_cal` + `rx_cal.rx_check`	RX Cal
DDMI_VCC, DDMI_Temp	`vcc_cal.vcc_cal` + `vcc_check`; `temp.read_temp`	VCC Cal
ShowPF	`showpf.cleanup`; pass/fail in `results.py`	Final
Move_out / MES	`runner.py`, `ddmi/mes`, `prod_csv.py`	–

Time budget

Production: 568 s of program time for two modules, about **5 min 19 s per module** including about 35 s of handling. Fixed waits add up to only about 50–60 s per module. The rest is I²C: IIC.vi waits **500 ms after each page switch** on many reads (R_RSSI, R_DRXP…R_IBias, R_Bias_setpoint…R_tops setpoint). This was raised from 75 ms in 2025-09. Python now waits the same per command (section 13), so a run takes about as long as V6.8's; it waits longer only in the sweep and Scan_Disable, where this station needs it.

1. Start-up

Purpose	Bring the bench and the module to a known state, check the SN, trim the supply so the module sees 3.3 V, check that it draws current.
V6.8	`800G_TRx_Test.vi` Start event queue: **Reini, Set_Temp, Read SN, Move_in (disabled), Read_Spec, Wait_Temp, VCC**, StartTest; `Auto Adjust Vcc.vi`, `I2C Read Vcc.vi`, `PS Read Current.vi`
Python	`ddmi/steps/startup.py`: `reini, set_temp, read_sn, wait_temp, set_vcc` (same order, `flow.run`); preceded by `flow.take_backup` PY
Config	`vcc:` nominal_v 3.3, adjust_wait_s 2.0, adjust_tolerance_v 0.01, current_limit_ma 6000, icc_min_ma 1800; `wait_temp:` enabled false
Results	ICC(mA), PN / rev / SN in the record

> **Power.** V6.8 never switches the supply on at DDMI Cal: `PS on-off.vi` sits only in the FQC / Final-test branch of Reini, and that is commented out. The operator powers the module, and Set_Temp and Read SN already talk to it before the VCC state trims the voltage. Python does the same: the run refuses to start when the supply output is off, and the driver never switches it.

Steps (in V6.8 order)

#	State	Action	Register / instrument	Wait
1	Reini	Every attenuator channel to **1.0 dB** (OVA Init on first use, OVA Set Chs Att). At DDMI Cal: no module write, no reset (W_TestMode / W_Reset only for FQC / Final test), no DCA.	AT4208 Set ATT	0.25 s per Set (driver)
2	Set_Temp	Station DDMI CAL, customer AWS: **AWS_Test_Mode = 0**; VCC target ("1V Select") = 3.3. No temperature is set.	W_AWS_Test_Mode EEh:80h = 0 HW	–
3	Read SN	Read PN (bytes 148–163), rev (164–165), SN (166–181); SN must equal the scanned SN	R_Information 00h:94h, 34 B	–
4	Read_Spec	Show the limits (the MES wrapper already downloaded them)	–	–
5	Wait_Temp	**Skipped at DDMI Cal** (the frame compares the station name with "DDMI Cal")	–	–
6	VCC	Auto Adjust Vcc to 3.3 V (algorithm below), current limit 6000 mA	supply + USB-ISS ADC	**2000 ms** per change
7	VCC	PS Read Current: must be **greater than** `ICC Spec.txt` = **1.8 A**	supply current	–

Auto Adjust Vcc.vi (automatic mode)

1. Read the module voltage (I2C Read Vcc.vi + Vcc Offset). It must be **2.8–4.0 V**, otherwise "Please check your Multimeter".
2. First setting = present supply output (PS Read Volt.vi) + (target − module) ÷ 2. It must be below 4.0 V.
3. Loop: set the supply (PS Set V & A, 6000 mA), wait **2000 ms**, read the module voltage. Done when |target − module| **< 0.01 V**. Otherwise setting += (target − module) ÷ 2 (**half-step** correction). Stop with a failure if the next setting is above 4.0 V.
4. Drop VCC = supply output − module voltage is shown on screen. Times = number of loops.

With about 0.3 V of cable drop, the half-step needs about 5 loops (about 10 s) per voltage. The same VI is used for the VCC cal points (section 7).

Limits / failures

- SN mismatch → "Inconsistent internal and external SN" → FAIL, stop.
- Module voltage outside 2.8–4.0 V → "Please check your Multimeter".
- Adjust fails → "VCC voltage regulation failed".
- ICC not above 1.8 A → "Current x.xxA failed" (module not drawing power or not seated).

Why AWS_Test_Mode must be 0 HW: with 1, the module acknowledges IBias writes but ignores them, and it freezes disabled lanes. TX Cal and Scan_Disable then fail. This was the root cause of the early failing runs.

V6.8 vs Python (aligned 2026-10-19: order, half-step adjust, 2.8–4.0 V pre-check, 4.0 V cap, 6000 mA limit, ICC strictly above 1.8 A, same messages)

- PY Before the first write, Python reads every register the run will change and saves it to `results/backups/<SN>_<time>_backup.json`. Restore it with `python -m ddmi restore --file …`. Read-only.
- PY Python reads the PN before MES Read_Spec, so MES returns the limits for this product.
- PY Extra adjust guards: at most 20 loops (V6.8 has no count limit), never more than 0.5 V above the target, and the supply driver's own station limit (3.9 V).
- PY The current limit is sent once per adjust. V6.8 sends it with every voltage, but the value never changes.
- PY Reini also sets the TX meter wavelengths (1270/1290/1310/1330 nm × 2) and its averaging (100 ms). The LX meter needs them.
- Reini uses 1.0 dB, or the station's minimum attenuation if that is higher (receiver protection on this bench).
- Python's `wait_temp` exists but is disabled (`enabled: false`), matching V6.8.

The module's own VCC monitor lags a supply change by seconds HW, so the 2 s wait is required. With 0.2 s, the VCC cal read the previous point.

2. MPD_Scan

Purpose	Find each lane's operating point on its modulator (MZM) curve, teach the firmware TOPS loop to hold it (tops_setpoint), and find the dark point (tops_off) used for TX disable.
V6.8	`800G SIP Tx MPD Monitor.vi` (frames 0–5) + `Peak Search.vi`; called from the Scan_MPD state (inner loop of 2)
Python	`ddmi/steps/mpd_scan.py`: `scan_mpd`, `_sweep`, `_analyse_and_write`, `_vtops_not_above_off`; maths in `ddmi/calc/tops.py`
Config	`mpd_scan:` in `config/product_aws_2xfr4.yaml`
Results	Tops_Setpoint, Tops_Mid(V), TXP_MPDOut_4.5dB / 3.5dB / 3dB / 2.5dB (record only)

What is measured

Signal	Register	Format	Meaning
Vtops	W/R E9h:A0h, 16 B	8 × U16, raw = V ÷ 5.0 × 4095	Heater voltage of the lane's phase shifter
TXMON	R F3h:86h, 16 B	8 × U16 little-endian HW	Monitor on the modulator's main output
MPD_OUT	R F3h:F0h, 16 B	8 × U16 little-endian HW	Monitor on the complementary output (falls when TXMON rises)

Lane map: laser *n* feeds CH*n* and CH*n*+4. Each of the 8 lanes has its own modulator, heater, TXMON and MPD_OUT.

Steps (MPD Monitor frame by frame)

Frame	Action	Registers	V6.8 wait	Python wait
0 Prepare	Bias loop off, then TOPS loop off	W_IBiasPID E9h:D0h = 0; W_TOPPID E8h:80h = 0	1 s	1 s (`pre_scan_wait_s`)
0 Prepare	Vtops of every lane = 1.0 V, tops_off of every lane = 0	W_Vtops_P; W_tops_off E8h:CCh = 0	1 s	1 s
1 Sweep	V = Vtops_Min + i · Step, 0.5 → 3.0 V (**126 points**), all 8 lanes together; read TXMON at each step; stop when V ≥ Vtops_Max. IIC.vi is called with **Delay = FALSE** (no 500 ms page wait)	W_Vtops_P, R_TXMON	**50 ms** per step	1 s at the first point, then **0.3 s** per step
2 Analyse	Per lane: Peak Search (peaks and valleys), pairing, target levels, Tops_Mid (below)	–	–	–
3 Operating points	For 4.5, 3.5, 3.0, 2.5 dB and Tops_Mid: write Vtops, wait, read TXMON and MPD_OUT	W_Vtops_P, R_TXMON, R_MPD_OUT	1 s per point	1 s (`op_point_settle_s`)
4 Write	Vtops = Tops_Mid; tops_off = valley voltage (both clamped to 0.2–3.0 V)	W_Vtops_P, W_tops_off	–	–
4 Write	Reset the module (brings TOPS and bias loops back on)	W_Reset 00h:1Ah = 08h	2 s before, **10 s** after	2 s / 10 s
4 Write	**After** the reset: tops_setpoint	W_tops setpoint E8h:82h, 8 × U16 big-endian HW	–	–
5 Read back	Scan passes when each lane's Vtops > its tops_off (AWS start room 0)	R_tops_Vtops, R_tops_off	–	–

The scan does not switch the loops back on itself: the block that writes TOPPID / IBiasPID = 255 is commented out, and the W_Reset in frame 4 restarts both loops. For AWS the **measured** setpoint is written and no bias setpoint. The fixed defaults (LC-2*FR4 900, MPO-DR8 320, Acacia 1500) belong to other customers.

Analysis: Peak Search and pairing (frame 2)

1. **Peak Search.vi**, run once for valleys and once for peaks. Threshold = **mean of the whole sweep**. Each run of points at or below the mean gives one valley (its lowest point), and each run at or above it gives one peak (its highest point). The first point wins a tie, and a run still open at the sweep end counts. A **peak run that starts at sweep point 0 is not reported** (the VI's run flag starts FALSE for peaks and TRUE for valleys). That is why a sweep starting high still pairs each valley with the peak after it.
2. **Pairing loop**: peak *k* with valley *k*. A pair is kept when all three hold:
   - peak index − valley index **> 1** (a distance in sweep points: the peak comes after the valley, i.e. a rising edge);
   - valley index **≥ 5** (AWS; default 20, Acacia 100);
   - peak index **<** (Vtops_Max − Vtops_Min) ÷ Step = 125 (the peak is not the last sweep point).
3. **Which pair**: AWS uses the default case, Array Max & Min on the kept valley amplitudes, **min index**, i.e. the pair with the **deepest valley**. Acacia takes the highest peak instead.
4. **Target levels**: constant `[0.65, 0.56, 0.50, 0.44]` used as (1 − x) × peak:

Constant	Level	dB below peak	Used for
0.65	35 % of peak	4.5 dB	TXP_MPDOut_4.5dB
0.56	44 % of peak	3.5 dB	TXP_MPDOut_3.5dB, Tops_Mid
0.50	50 % of peak	3.0 dB	TXP_MPDOut_3dB, Tops_Mid
0.44	56 % of peak	2.5 dB	TXP_MPDOut_2.5dB

5. **Closest step on the edge**: for each level, the sweep step between the chosen valley and peak whose TXMON is closest (Array Subset, |difference|, min index). No interpolation.
6. **Same-step rule**: if the 3.5 dB and 3.0 dB points land on the same step, the 3.5 dB point is set one step lower.
7. **Tops_Mid** = mean of the 3.5 dB and 3.0 dB voltages (inserted as the 5th point).
8. **Tops_setpoint** = round(TXMON ÷ MPD_OUT × 1000) at Tops_Mid (typical 500–900). **TXP_MPDOut_x** = the same ratio at the 4 levels (record only).

> **Checked on a real module** (P172262504894, our sweep vs the tops_off the LabVIEW station left): the chosen valley matches on 6 of 8 lanes within 0.05 V. CH3 and CH6 each have two valleys of equal depth (13 vs 14 and 10 vs 10 counts), so "deepest valley" depends on sweep noise. LabVIEW's sweep picked the other one. The old Python rule (first valley) matched 7 of 8 by chance but not by the same logic.

The sweep values are also converted to volts (/4096 × 2.5) for the charts and the Delta / Vavg displays. Peak Search, pairing and the closest-step search do not depend on that scale.

Loop / retry logic

- Inner loop (up to 2 scans): re-scan when the frame-5 check fails (Vtops not above tops_off).
- Outer loop (up to 2 passes of scan + Adjust Power + Scan_Disable): runs again only when Scan_Disable fails.

> **Why the rising edge matters** HW: on a falling edge, the firmware loop does not hold the lane after the reset. The lane sits at the top of the fringe (about +5 dBm, MPD_OUT ≈ 25), and that gave the "4 dB on CH1" readings seen earlier.

> **V6.8 vs Python** (aligned 2026-10-19: frame 0 order, Peak Search, same-index pairing with its 3 conditions, deepest valley, same-step rule)
>
> - Sweep step wait: **0.3 s** instead of 50 ms (first point 1 s). This station's TXMON refreshes only every ~200 ms, and 50 ms gave stepped curves. It is a config value (`mpd_scan.step_settle_s`), so 0.05 can be tried for a direct comparison.
> - PY Lane without a valid pair → rescan once; still none → that lane keeps its existing tops_setpoint and tops_off and fails. V6.8 indexes an empty array (zeros), writes 0.2 V to both registers, and the read-back check (Vtops > tops_off) fails.
> - PY Flat curve (peak < max(2 × valley, valley + 50)) → lane kept, logged.

- PY After the write, `tops_lock.relock` checks every lane is locked (section 12).
- The sweep is drawn live in the UI (MPD graph) from `mpd_point` events.

3. Adjust Power

Purpose	Set the laser bias set-points to the AWS defaults and nudge each laser once so the TX power lands in a window.
V6.8	`Item_Test.vi` case "Adjust Power" (AWS LC-2*FR4); outer Scan_MPD loop counter = 0 and global `Only_Test` = False
Python	`mpd_scan.adjust_power` (first pass only)
Config	`adjust_power:` default_setpoint [280, 280, 410, 450], window_dbm [0.7, 2.3], step_up 20, step_down 30, settle_s 2.0, min_light_dbm −10
Results	Bias_setpoint (read in TX Cal)

#	Action	Register	Wait
1	Write default [280, 280, 410, 450] (one value per laser) to all three set-points	W_Bias_setpoint RT E9h:D2h, HT E9h:B0h, LT E9h:B8h (IIC.vi 55/56/57) HW	–
2	OSW Switch CH (no wait at this station: its 2 s delay runs only for switch types such as EOS-108, not for "Not Used" / WG3039), then OPM Read Chs Power. True power = meter + Tx Loss[ch]	power meter	**2 s** before
3	Per laser *i* (loop of 4): mean (Mean.vi, in dBm) of lanes *i* and *i*+4. In Range 0.7–2.3 dBm (limits included) → 0; otherwise below **0.7 dBm** → **+20**, above → **−30**. **One step only.**	–	–
4	Only if the new array differs from the default: write it to RT, HT and LT (no wait after)	same three registers	–

Verified 2026-10-19 against the Item_Test.vi "Adjust Power / AWS / LC-2*FR4" wiring, node by node. `mpd_scan.adjust_power` already did the same steps in the same order, so no code change was needed.

A module that already passed TP2/TP3 and is re-run through DDMI Cal gets its bias set-points reset to these defaults. Its current and TX power drop (seen: about 150 mA less, lower power). The final power tuning (up to 5 steps, 150–750) is done at the TP2/TP3 station.

V6.8 vs Python

- PY Light guard: if any lane reads below −10 dBm *before* the write (TX fibre off, meter not connected), the run stops and the bias is left unchanged. This was added after −79 dBm on every lane still changed the set-points.
- PY The pre-run backup includes RT/HT/LT, so the old set-points can be restored.
- The MPO-DR8 branch ([300, 350], window 1.0–2.6 dBm) is not ported (not used for 2×FR4).

4. Scan_Disable (TX disable)

Purpose	Tune each lane's tops_off (heater voltage used when the lane is disabled) so that a disabled lane is as dark as possible.
V6.8	`Item_Test.vi` case "Scan_Disable" / Siph (frames 0–5; the 7-step block in frame 1 and the index block in frame 2 are commented out)
Python	`mpd_scan.scan_disable`, `_measure_disabled`, `_wait_dark_reading`
Config	`tx_disable:` step_v 0.01, steps 4, recheck_dbm −18, extra_steps 5, enable_wait_s 1.0, limit_dbm [−60, −16], settle poll (Python)
Results	Tx_Disable(dBm), Tops_off(V), Tops_Mid

One measuring step (error-wire order)

#	Action	Register / instrument	V6.8 wait
a	Write tops_off for all 8 lanes	W_tops_off E8h:CCh	–
b	Disable all lanes (the module applies tops_off as a lane goes into disable) HW	W_TXDis 10h:82h = FFh (IIC.vi "channel 9")	**0 s**
c	Read all 8 powers (OPM Read Chs Power + Tx Loss)	power meter	–
d	Enable all lanes	W_TXDis = 00h ("channel 8")	**0.01 s**
e	Read back Vtops and tops_off (display only)	R_tops_Vtops, R_tops_off	–

Search (per lane, all lanes stepped together)

1. **Frame 0**: start = present tops_off (the scan's valley). Measure **start, start + 0.01, start − 0.01**. Direction per lane: **up if (start + 0.01) is darker than start, otherwise down**. +0.01 and −0.01 are not compared with each other.
2. **Frame 1**: 4 steps at (start ± 0.01) ± k · 0.01, k = 1..4, i.e. 2 to 5 steps away from the start. Every step is taken, whatever it reads.
3. **Frame 2**: per lane, the voltage with the lowest power of all 7 points (transpose, Array Max & Min, min index: first one on a tie).
4. **Frame 3**: write it; disable; wait **0.1 s**; read the leakage (this is the Tx_Disable result); enable; wait **1 s**.
5. **Frames 4–5**: if **any** lane reads above **−18 dBm**, **every** lane takes 5 more steps (k = 5..9), the darkest of all 12 points is written and frame 3 is repeated.

Limit: **−60 to −16 dBm**. −18 dBm is only the trigger for the extra steps. If a lane fails, the outer Scan_MPD loop runs once more.

> HW A tops_off write to a lane that stays disabled has no effect. The lane must go from enabled to disabled, which is why every step disables all lanes and enables them again.

> **Aligned 2026-10-19.** Python now uses the V6.8 step order (write, disable, read, enable, 0.01 s) and its direction rule. It measures the same fixed points (4 steps always taken), picks the darkest of all points, and gives every lane the 5 extra steps. Before, Python compared +0.01 with −0.01, only moved when a step was darker (so it could repeat a point), and stepped only the bright lanes.

> **V6.8 vs Python**
>
> - Settle before each reading: V6.8 reads straight after the disable (0 s; 0.1 s for the final reading). On this station the leakage needs seconds to settle, so Python polls every 0.5 s (min 1 s, max 8 s). A reading counts once 3 readings agree within 0.5 dB, or the lane is below −22 dBm. The log lists every wait.
> - PY The lanes are re-enabled in a `finally` block, even on abort or a meter error.
> - PY TOPS relock check after Scan_Disable.

5. TX Cal (DDMI_Adjust_TX)

Purpose	Calibrate the module's TX-power monitor: per lane, a slope and offset so that the reported power = true power.
V6.8	`Item_Test.vi` "DDMI_Adjust_TX" (retry loop, record) → `DDMI_Cal.vi` "Txp" / "Cal" (sequence 0: measure and fit; sequence 1: restore and reset) and "Check"; `Calulate_Curve Fit Coefficient.vi` (linear)
Python	`ddmi/steps/tx_cal.py`: `tx_cal`, `tx_check`; fit in `calc/fit.py`
Config	`tx_cal:` bias_ma [60, 180], settle_s 2.0, after_reset_wait_s 10.0
Results	Power(dBm), DDMI_TxP, dTxP, Txp_Offset, Txp_Slope, Bias_setpoint, Tops_setpoint

Cal (DDMI_Cal.vi "Txp" / "Cal", Siph)

#	Action	Register / instrument	Wait
1	Read and keep the present bias of the 4 lasers	R_IBias E9h:C0h, 4 × U16 = mA × 100 HW	–
2	For **60 mA** then **180 mA** (TxP_Cal, AWS LC-2*FR4): bias loop off; bias on all 4 lasers	W_IBiasPID E9h:D0h = 0; W_IBias	**2 s**
3	Read the 8 true powers (OPM Read Power + Tx Loss[ch]), then TXMON	meter; R_TXMON F3h:86h	–
4	Per lane: x = TXMON, y = 10^(dBm/10) × 10000 (**not rounded**) → linear fit → slope, offset; write per lane	W_DTXPCal: ECh (CH1–4), EDh (CH5–8), offsets C0h, slopes D0h	–
5	Bias loop on, then the kept bias back, then reset	W_IBiasPID; W_IBias; W_Reset	**10 s**

Check and record (DDMI_Cal.vi "Txp" / "Check", Item_Test.vi "DDMI_Adjust_TX")

1. At normal bias after the reset: true power (OPM + Tx Loss) and R_DTXP 11h:9Ah (8 × U16, 0.1 µW); dTxP = true dBm − DDMI dBm. R_DTXPCal is read for the display.
2. Pass = every lane Txp_Slope in **5–150** (In Range) and |dTxP| **< 0.5 dB**. Cal + Check run again once if not (loop counter ≥ 1 ends), 50 ms apart.
3. Finally R_Bias_setpoint and R_tops setpoint are read and recorded (Bias_setpoint, Tops_setpoint).
4. The AWS bias-setpoint adjustment block inside DDMI_Adjust_TX (window 1.2–2.2 dBm, 400–1400, step 200) is in the **disabled** subdiagram of a Diagram Disable structure.

Bias points by product: LC-2*FR4 AWS 60/180 mA; LC-2*FR4 other 70/130; MPO-DR8 100/200, AWS 100/300; Acacia 80/120. Lanes *n* and *n*+4 share a laser but have their own TXMON, so all 8 lanes get their own fit.

> **Aligned 2026-10-19.** The fit now uses the unrounded linear power (it was rounded to an integer). The restore order is now W_IBiasPID, W_IBias, W_Reset (Python wrote the bias first). Tops_setpoint is now recorded from the register read at the end, as V6.8 does. Bias points, waits, fit, write layout, check and the one re-run already matched.

> **Closed 2026-10-20: the IBiasPID value.** In sequence 1, V6.8 passes a 1-element DBL array constant to W_IBiasPID: **15.0** for LC-2*FR4 (stored as 402E000000000000), 3.0 for MPO-DR8. The 16430 (0x402E) and 16392 (0x4008) read earlier were only the first 2 bytes of those numbers. V6.8 writes 15, which is what a LabVIEW-finished module reads at E9h:D0h. Python writes back the value read before the run (15), so both match.

V6.8 vs Python

- PY If TXMON hardly moves between 60 and 180 mA (< 20 %), the fit would be noise. That lane keeps its old calibration and fails.
- PY The bias, IBiasPID and reset are restored in a `finally` block (also on abort).
- PY Before the run starts: Tx Loss pre-flight. The run stops if Tx Loss is missing, and gives a warning above 3.5 dB.

- PY TOPS relock check after the reset. A lane the scan could not set keeps its failed (empty) Tops_setpoint instead of the register value.

Tx Loss is part of the TX calibration. True power = meter + Tx Loss[ch], so any error in Tx Loss goes 1:1 into every module's TX calibration. It must be measured on this bench's own fibres with the golden sample (section 11).

6. RX Cal (DDMI_RXP)

Purpose	Calibrate the module's RX-power monitor (RSSI) per lane from two known input powers, then check it at a third.
V6.8	`Item_Test.vi` "DDMI_RXP" (retry loop + check) → `DDMI_Cal.vi` "Rxp" / "Cal" (its "Check" case is empty); `CH RxP To Att.vi`, `Read ATT Table_400G.vi`, `OVA Set Wavelength.vi`, `OVA Set Chs Att.vi`, `Calulate_Curve Fit Coefficient.vi`
Python	`ddmi/steps/rx_cal.py`: `rx_cal`, `rx_check`, `_set_power`, `_pairs`, `_pair_wait_s`; tables in `calc/rxtable.py`
Config	`rx_cal:` points_dbm [−2, −10], check_dbm −6, wavelengths_nm, att_delay_s 3
Results	RxP1, DD_RxP1, dRxP1, Rxp_Offset, Rxp_Slope

Both the calibration and the check use the enabled "2XFR4 two CH Parallel" subdiagram (the one-lane-at-a-time variant is in the disabled frame of a Diagram Disable structure). Lane pairs (1,5), (2,6), (3,7), (4,8): lanes 1–4 are on RX fibre 1 (attenuator channel 1), lanes 5–8 on RX fibre 2 (channel 2).

Calibration (DDMI_Cal.vi "Rxp" / "Cal")

#	Action	Register / instrument	Wait
1	For −2 dBm, then −10 dBm (`RxP_Cal`), and for each pair i = 0..3:		
2	Table lookup for lane i and lane i+4 (CH RxP To Att → Read ATT Table_400G, linear interpolation)	`CHn_Rx_calibration.txt`	–
3	Set Wavelength for both lanes; one Set ATT for both attenuator channels ("two CH Parallel")	AT4208 / OVA	0.25 s per Set (driver)
4	Wait **"ATT Delay(s)" + 2 s**	station setting	**5 s** with the saved 3
5	Read RSSI; keep lanes i and i+4	R_RSSI F3h:A6h, 8 × U16 LE	500 ms page switch (IIC.vi)
6	Per lane: y = 10^(dBm/10) × 10000 (**not rounded**: 6309.57 and 1000), x = RSSI → linear fit → slope, offset; write per lane	W_DRXPCal: ECh / EDh, offsets 90h, slopes A0h	–

Check and retry (Item_Test.vi "DDMI_RXP")

1. Read back the calibration (R_DRXPCal) for the display.
2. Check at **−6 dBm** in the same pairs: lookup, Set Wavelength × 2, Set ATT both, wait "ATT Delay(s)" + 2 s, read **R_DRXP** 11h:BAh. dRxP = DDMI dBm − (−6).
3. Pass = every lane |dRxP| **< 0.5 dB** and every Rxp_Slope in **5–30** (In Range). The loop stops when it passes or after the second run (loop counter ≥ 1); each run is Cal + Check; 50 ms between runs.

> Lanes on one RX fibre share an attenuator channel, so each lane's own setting only holds while its pair is being read. That is why the check also runs pair by pair.

> **"ATT Delay(s)"** is a field on the Set tab of 800G_TRx_Test.vi (whole seconds). The value saved in the VI is **3**, so the V6.8 wait is 5 s per pair, and Python uses 3. At start-up V6.8 replaces it from `system_Data.ini` (key Device), which is not in our copy, and the saved panel is not this station's setup (MPO-DR8, EXFO). Please confirm the value on the production DDMI Cal station's Set tab.

> **Aligned 2026-10-19.** The wait after each pair is now att_delay_s + 2 s (it was a flat 1.0 s), and the fit uses the unrounded linear power (it was rounded to an integer). Pairs, order, check point, limits and the one re-run already matched.

> **V6.8 vs Python**

- This bench uses a TriBrer **AT4308** (32-bit DLLs through an x86 host) instead of the Semight AT4208. Wavelength and attenuation are set per lane, so a pair takes two Set ATT calls instead of one parallel call. The effect is the same.
- PY If RSSI does not at least double between −10 and −2 dBm (6.3× in power), that lane keeps its old calibration and fails.
- PY A missing RX table stops the run before any write.
- `python -m ddmi rx-settle` measures how fast RSSI actually settles on this bench, for comparison with the V6.8 wait.

7. VCC Cal (DDMI_VCC)

Purpose	Calibrate the module's supply-voltage monitor from two supply voltages, then check it at 3.3 V.
V6.8	`Item_Test.vi` "DDMI_VCC" (retry loop) → `DDMI_Cal.vi` "Vcc" / "Cal" and "Check"; `Auto Adjust Vcc.vi`; `I2C Read Vcc.vi`
Python	`ddmi/steps/vcc_cal.py`: `vcc_cal`, `vcc_check`, `needs_rerun`; `startup.auto_adjust_vcc`
Config	`vcc:` cal_points_v [3.13, 3.47], settle_s 2.0, clear_wait_s 1.0, check_wait_s 1.0, nominal_v 3.3, current_limit_ma 6000
Results	I2C_Vcc, DDMI_Vcc, dVcc, Vcc_Offset, Vcc_Slope (module-level, shown in all 8 columns)

Cal (DDMI_Cal.vi "Vcc" / "Cal", error-wire order)

#	Action	Register	Wait
1	Unlock: DSP debug, then dual mode	W_Debug_AWS CFh:81h = 63; W_Daul_AWS CFh:86h = 64, then CFh:80h = 63	–
2	Clear the old calibration	W_VccCal EBh:90h = [0, 0]	"Time Delay2" (time not wired; LabVIEW default **1 s**)
3	For **3.13 V** then **3.47 V**: Auto Adjust Vcc (Vcc Offset 0, 6000 mA), wait, read the VCC ADC	supply + USB-ISS ADC; R_VCCADC F3h:84h	2 s per supply change; then **2.0 s**
4	x = ADC, y = **measured** module voltage (Auto Adjust's I2C reading) × 10000 → linear fit → slope, offset; write	W_VccCal: offset EBh:90h, slope EBh:94h (float32 LE)	–
5	Lock: normal, then mission mode	W_Nomal_AWS CFh:81h = 0; W_Mission_AWS CFh:86h = 0, then CFh:80h = 63	–

Check (DDMI_Cal.vi "Vcc" / "Check", AWS)

1. Auto Adjust Vcc to **3.3 V** (Item_Test passes FALSE to the "skip adjust" input, so the adjust runs), then "Time Delay10" (time not wired: **1 s** assumed).
2. Read **R_DVCC** 00h:10h (U16 × 100 µV), then read the module voltage **again** (I2C Read Vcc), then R_VccCal (display).
3. dVcc = module voltage − DDMI voltage, in **volts** for AWS. The default case (other customers) gives a percentage (difference ÷ voltage × 100). The production CSV still names the column "dVcc(%)".

Retry (Item_Test.vi "DDMI_VCC")

Cal + Check run again only when **|dVcc| > 1.0** (V for AWS; the loop stops on |dVcc| ≤ 1.0 or after the second run), at most twice in all, 50 ms apart. The slope and the dVcc limit are **not** part of the retry; they are judged at the end (ShowPF) like every other item.

Reference voltage (I2C Read Vcc.vi)

Since 2025-10-10, V6.8 reads the module voltage through the USB-ISS adapter's ADC behind the 3.3 V divider: init `5A 02 60 0F`, set pins `0x63`, `GET AnalogData 0x65`, 2 bytes back. V = (b0·256 + b1) × Vref ÷ 1024 × Scale. Python reads the same USB-ISS analogue pin (`power_supply.vcc_monitor` in the station config).

Limits

I2C_Vcc 3.234–3.366 V, dVcc **±0.033 V**, Vcc_Slope **15–30**.

> **Aligned 2026-10-19.** Added the wait after clearing VccCal and after the check's adjust. The check now reads the module voltage again after R_DVCC (it used the adjust's last reading). The re-run now follows V6.8 (|dVcc| > 1.0;

strict since 2026-10-20); before, Python re-ran on any Vcc_Slope or dVcc limit failure. Order, points, waits, fit input and the lock/unlock registers already matched.

V6.8 vs Python

- The two unwired waits ("Time Delay2", "Time Delay10") use LabVIEW's 1 s default. Their configured time is stored inside the express VI instances and could not be read from the copy. They are config values (`clear_wait_s`, `check_wait_s`).
- PY The old calibration is read first. If the fit fails (ADC changes by < 20 counts), the old calibration is written back instead of leaving 0/0.
- PY The DSP modes are restored in a `finally` block, to the values read before the run (0/0).

8. Temperature (DDMI_Temp)

Purpose	Record the module temperature and check it is plausible.
V6.8	`Item_Test.vi` "DDMI_Temp": one IIC.vi R_DTEMP read, shown in all 8 columns with 2 decimals. It does **not** call DDMI_Cal.vi "Temp" (that case holds the temperature-offset calibration for other stations).
Python	`ddmi/steps/temp.py`: `read_temp`
Results	DDMI_Temp, limit **30–60 °C**, no retry

Read R_DTEMP 00h:0Eh, signed 16-bit ÷ 256 = °C (e.g. 2A80h → 42.5 °C). **No temperature calibration at DDMI Cal.**

Verified 2026-10-19 against Item_Test.vi "DDMI_Temp" (5 nodes: IIC.vi, a loop of 8 for the display). Python already matched; no change.

For reference, DDMI_Cal.vi "Temp" / "Cal" (not used here) reads the case temperature (TM Read Temp) and R_DTEMP 3 times per pass, 500 ms apart, 2 passes, and writes the mean offset to W_TempCalRT / HT / LT. Its "Check" case uses fixed case temperatures for Final test LT (−2.5 °C) and HT (72.5 °C).

9. ShowPF (final clean-up and PASS/FAIL)

Purpose	Put the module back into its shipping state and judge every result against its limit.
V6.8	`800G_TRx_Test.vi` state ShowPF, customer AWS, station "DDMI CAL / DDMI Cal" (frames 0–3); `Dispaly.vi` for the result display
Python	`ddmi/steps/showpf.py`: `cleanup` (mode `labview`, default); pass/fail in `ddmi/results.py`

Frame 0: temperature-offset function, up to 3 tries

#	Action	Register	Wait
1	Disable Temp Off Func	FEh:**95h** = **59** (FEh:98h only for type Oracle(AWS DUT), material 400454000050)	**2 s**
2	Read Temp Off Func; stop when it reads 59, else try again (loop counter ≥ 2 ends: 3 tries)	same	(200 ms runs in parallel)

Frame 1: test mode, reset, DSP modes, up to 3 tries

#	Action	Register	Wait
1	Exit test mode	W_TestMode EEh:83h = CCh	–
2	Reset	W_Reset 00h:1Ah = 08h	**10 s**
3	Read TestMode, DSP operating mode, DSP test mode	R_TestMode EEh:83h; R_DSP_OperatingMode CFh:86h; R_DSP TestMode CFh:81h	–
4	Stop when they read CCh, 0, 0; else repeat 1–3 (3 tries, each with its own reset and 10 s)		(200 ms in parallel)

After the loops: "Disable Temp Offset Fail!", "Test Mode Set Fail!" or "DSP Mode Reset Fail!" for whichever check is still wrong (the item fails).

Frames 2–3: PASS/FAIL and next module

Every result is compared with its Low/High limit (`Dispaly.vi`, PSM4_State_FGV P/F); failing items are listed per lane as "CH*n_item*:value". Frame 3 rebuilds the state list for the next temperature/module.

Confirmed 2026-10-19 on a module finished by the LabVIEW station (P172262504894, `regcheck`): TempOffFunc 59, TestMode CCh, DSP 0/0, AWS_Test_Mode 0, IBiasPID 15, TOPPID 255 HW.

> **Aligned 2026-10-19.** The temp-offset loop now waits 2 s before each read-back and tries up to 3 times (was 200 ms, 2 tries, then one 2 s wait). TestMode is now read **after** the reset, and the reset + 10 s + DSP check are inside the TestMode retry loop (3 tries). Before, Python read TestMode before the reset, retried only the write, and reset once.

> **V6.8 vs Python**
>
> - PY Final TOPS lock check (`final=True`). A lane still unlocked after the relock attempts fails the run. The TOPS_Relock count per lane is recorded.
> - PY The module driver re-sends the vendor password after each reset (needed to read the vendor pages again).
> - PY Optional `mode_cleanup: restore` writes back the exact pre-run mode registers instead.

10. MES, records and data storage

V6.8	`Cimes_Interface.vi` → `Cimes_All.vi` → `Mes Interface.vi` / `Get Soap content.vi`; `Table Value to MES Data.vi`; `Save Record.vi`, `Save Picture.vi`, `Timing.vi`
Python	`ddmi/mes` (SOAP + OfflineMes), `ddmi/runner.py`, `ddmi/prod_csv.py`, JSON records in `results/records`

MES

Item	Value
Server	`http://172.30.1.139/CIMES_CN_PRD/AOIRule/AOIServices/LabViewService.asmx` (from `MesServer Address.txt`), station "DDMI Cal"
Before the test	Pcs_Check_IN (route check; failure → no test), Read_Date, Read_Spec (limits for PN + station; override the built-in ones), Read_Test_Times, TOOL
After the test	Save_Data = InsertTestDataByPcs (table TestResult_800G_TRX_TEST), retried up to **3** times; **Pcs_Check_OUT only if Save_Data succeeded**
Reply check	regex `(?<=<Result>).*?(?=</Result>)` must be "OK", otherwise "MES Return abnormal content!"
Record format	Table Value to MES Data.vi: per channel `<n>_DDMI,Pass|Fail,<error item>,v1,v2,…`; a value that is NaN, −Inf or 20+ characters is sent as 0; channels joined by `$`. Python builds the same string (`ddmi/mes/record.py`); the value columns follow the production CSV order (LabVIEW's table column order is not in the copy)
DUT type	AWS 2×FR4 = material 400454000030 → type AWS (not Oracle)

> **Without MES access** (as now) the run uses `OfflineMes`, like V6.8's "Local" mode: check-in and check-out always succeed; limits come from the product config (or `results/mes_offline/spec.json` if present); Save_Data writes a JSON file to `results/mes_offline/outbox/` holding the full record and the exact `mes_data` string V6.8 would upload. Save_Data is tried up to 3 times; Pcs_Check_OUT only follows a successful save. **Aligned 2026-10-19**: the 3 tries and the V6.8 record string were added. The online SOAP client is not written yet: it needs MES access to test.

Local files

Data	V6.8	Python
Daily CSV	`Record\%Y-%m-%d_2XFR4 Record.CSV` (append); production format `YYYY-MM-DD_DDMI2XFR4.csv`: MAX row, MIN row, CH1–CH8 rows	`prod_csv.py`, same columns, appended after every real PASS or FAIL run to `Desktop\ddmi_python_test` (temporary location, `production_csv_dir`)
Picture	`\\172.30.1.164\TR…\Picture\%m-%d\`	– (log + JSON record instead)
Time stamp	Timing.vi `%Y-%m-%d-%H-%M-%S`	ISO time in the record
Full record	–	`results/records/<SN>_<time>_slot<n>.json` (all values, log, MPD curves)

Result columns (Step_To_Item.vi, AWS LC-2*FR4)

Group	Columns	Limits
Scan_Disable	Tx_Disable(dBm), Tops_Mid, Tops_off	−60–−16; 0–3; 0–3
DDMI_Adjust_TX	Power, DDMI_TxP, dTxP, Txp_Offset, Txp_Slope, Bias_setpoint, Tops_setpoint	≥−3.3; ≥−3.8; ±0.5; ±10000; 5–150; 100–2500; 100–2500
DDMI_RXP	RxP1, DD_RxP1, dRxP1, Rxp_Offset, Rxp_Slope	–; –; ±0.5; ±10000; 5–30
DDMI_VCC	I2C_Vcc, DDMI_Vcc, dVcc, Vcc_Offset, Vcc_Slope	3.234–3.366; –; ±0.033; ±10000; 15–30

DDMI_Temp	DDMI_Temp	30–60
TXP_MPDOut	_4.5dB, _3.5dB, _3dB, _2.5dB	record only

Python `results.limits_from` accepts a MES spec keyed by config key, by result item name or by production column name.

11. Station calibrations (CAL tab, not part of each test)

Tool	V6.8	Python
Tx Loss Cal (not aligned to V6.8: this bench follows the production Tx Loss tool)	Golden sample: Tx Loss[ch] = true power − station reading. Saved as one tab-separated line CH1→CH8 in `system_Data.ini` [Data] `Tx Loss`. Warning above 3.5 dB (2×FR4).	Tx Loss tool in the UI. Waits for the golden sample's temperature to settle (a reading every 15 s, 3 in a row changing by < 0.1 °C, max 15 min).
Rx Loss Cal (RX tables)	`0-30ATT.vi` (prompts "Connect the DUT Rx1 / Rx2 input signal to the optical power meter!", file `CHn_Rx_calibration.txt`) → `Att Calibration.vi`: all 8 attenuator channels at 40 dB; the lane's channel stepped Start 0 → Stop 20 in 1 dB steps, a 0 replaced by 1 (so 1, 1, 2 … 20: 21 points); wait **"ATT Delay(s)" + 0.5 s**; read the meter; row = attenuation, −power (2 decimals); at the end every channel back to **1 dB**.	RX Attenuation Table tool (`tools/rx_table.py`): same prompts, other fibre at 40 dB, the same 21-point sequence, wait `rx_cal.att_delay_s` + 0.5 s, attenuators back to 1 dB (or the station minimum) at the end; same file format, read by `calc/rxtable.py`. Python also sets the lane's wavelength (the AT4308 needs it). **Aligned 2026-10-20** (was 20 points, 0.5 s, no reset to 1 dB).

> Both tables feed straight into the calibration. A Tx Loss copied from another bench, or an RX table from other fibres, shifts every module's calibration by the same error. Measure both on this bench, with its own fibres and the golden sample.

12. Python-only safety additions

Addition	Where	Why
Register backup before the first write	`flow.take_backup`, `backup.py`	Undo a run (`python -m ddmi restore`)
Register write guard	`hw/regmap`, `registers_aws.yaml`	Writes only to registers confirmed on hardware (all are now confirmed)
TOPS relock check	`steps/tops_lock.py`	One lane was seen sitting at the fringe top after the loop came on. Check the ratio is within 30 % of the set-point (MPD_OUT ≥ 100); else loop off, Vtops = Tops_Mid, loop on (2 tries). Final check fails the run.
Safe state on abort / error	`flow.safe_state`	TX enabled, reset (loops on), supply to nominal. Reports any action that failed.
Fit sanity checks	TX, RX, VCC cal	A fit on readings that hardly moved keeps the old calibration instead of writing noise
Light guard	`adjust_power`	No bias change without real light
Tx Loss pre-flight	`runner.py`	No run without a Tx Loss file; warning above 3.5 dB
Read-only tools	`python -m ddmi regcheck`, `rx-settle`	Compare a module with the V6.8 end state; measure RX settling
Shared instruments	`hw/shared.py`	One meter / attenuator safely shared by several slots

13. All delays in one table

\n\1

Section	Where	V6.8	Python
1, 7	Auto Adjust Vcc, after every supply change (half-step, ~5 loops)	2000 ms	2.0 s
1, 6, 11	Attenuator after Set ATT / Set Wavelength	0.25 s (AT4208 driver; read: 0.02 s)	0.25 s (AT4308 driver)
2	Frame 0, after Vtops = 1.0 V and after tops_off = 0	1 s + 1 s	1 s + 1 s
2	Sweep, per step (126 steps)	50 ms	0.3 s (first point 1 s)
2	Each operating point (5)	1 s	1 s
2	Before / after W_Reset	2 s / 10 s	2 s / 10 s
3	After the default bias write	2 s	2 s
4	Scan_Disable: read after disable / after enable / final read / after final enable	0 s / 0.01 s / 0.1 s / 1 s	settle poll 0.5 s (1–8 s) / 0.01 s / settle poll / 1 s
5	Per bias point / after W_Reset	2 s / 10 s	2 s / 10 s
6	After setting a lane pair (cal × 8, check × 4 per run)	"ATT Delay(s)" + 2 s (5 s with the saved 3)	att_delay_s + 2 s (5 s)
7	VCC: after clearing VccCal / after each adjust point / after the check adjust	1 s* / 2 s / 1 s* (*unwired, default)	1 s / 2 s / 1 s
5–7	Before re-running a failed item	50 ms	50 ms
9	ShowPF: Temp Off Func write → read (≤ 3 tries) / TestMode + W_Reset → read (≤ 3 tries)	2 s / 10 s per try	2 s / 10 s per try
all	IIC.vi after the page select of every command	500 ms (R_RSSI, R_DRXP…R_IBias, R_Bias_setpoint…R_tops setpoint reads); 75 ms (all other commands); none in the MPD sweep (Delay = FALSE)	the same (`Module.PAGE_WAIT_S`, `SLOW_READ_WAIT_S`; off in the sweep and the simulator)
–	OSW Switch CH (where an optical switch is used)	2 s	not used on this bench
11	Rx Loss Cal, per attenuation point	\"ATT Delay(s)\" + 0.5 s	att_delay_s + 0.5 s

Checked 2026-10-20 row by row against the V6.8 VIs and the Python code. Added in this check: IIC.vi's page-select waits (500 ms / 75 ms per command, the largest part of V6.8's run time), 0.25 s after each attenuator Set, and 50 ms before a re-run. Two differences remain on purpose: the 0.3 s sweep step and the Scan_Disable settle poll (both needed on this station). Three V6.8 waits depend on values not in the copy: "ATT Delay(s)" (3 saved in the VI, production value to confirm) and the two unwired VCC Time Delays (1 s assumed).

14. Register master list (AWS)

Vendor pages 80h–FFh need a password at 00h:7Ah–7Dh: **'AOIW'** (41 4F 49 57) for reading, **83 4A 2D 4C** for writing. With only 'AOIW', vendor writes are acknowledged but discarded HW. Standard CMIS words and tops_setpoint are big-endian. Raw monitors on F3h and calibration floats are little-endian HW. All registers below are confirmed on hardware.

Register	Page:Byte	Len	Format / value	Section
R_Information	00h:94h	34	ASCII PN / rev / SN	1
R_DTEMP	00h:0Eh	2	S16 ÷ 256 °C	8
R_DVCC	00h:10h	2	U16 × 100 µV	7
W_Reset	00h:1Ah	1	08h	2, 5, 9
W_TXDis	10h:82h	1	FFh all off, 00h all on	4
R_DTXP	11h:9Ah	16	8 × U16, 0.1 µW	5
R_DBias	11h:AAh	16	8 × U16	record
R_DRXP	11h:BAh	16	8 × U16, 0.1 µW	6
R_VCCADC	F3h:84h	2	raw, LE	7
R_TXMON	F3h:86h	16	8 × U16 LE	2, 5
R_RSSI	F3h:A6h	16	8 × U16 LE	6
R_MPD_OUT	F3h:F0h	16	8 × U16 LE	2
W_TOPPID	E8h:80h	1	0 off; 255 after a LabVIEW run	2
tops_setpoint	E8h:82h	16	8 × U16 big-endian	2
tops_room	E8h:ACh	16	V ÷ 5 × 4095 (not used, AWS start room 0)	–
tops_off	E8h:CCh	16	V ÷ 5 × 4095	2, 4
Vtops	E9h:A0h	16	V ÷ 5 × 4095	2
Bias_setpoint HT / LT / RT	E9h:B0h / B8h / D2h	8 each	4 × U16, one per laser	3, 5
IBias	E9h:C0h	8	4 × U16 = mA × 100	5
W_IBiasPID	E9h:D0h	1	0 off; 15 on	2, 5
VccCal	EBh:90h	8	offset @90h, slope @94h, float32 LE	7
DTXPCal	ECh / EDh:C0h	32 each	4 offsets then 4 slopes, float32 LE	5
DRXPCal	ECh / EDh:90h	32 each	4 offsets then 4 slopes, float32 LE	6
AWS_Test_Mode	EEh:80h	1	0 at DDMI Cal	1
TestMode	EEh:83h	1	CCh to exit	9
DSP TestMode (Debug / Nomal)	CFh:81h	1	63 / 0	7, 9
DSP OperatingMode (Daul / Mission)	CFh:86h	1	64 / 0	7, 9
DSP command (written by W_Daul_AWS and W_Mission_AWS after CFh:86h)	CFh:80h	1	63	7
Temp Off Func	FEh:95h	1	59 (Oracle: 98h)	9

Checked 2026-10-20 against IIC.vi's AWS case for every command (page and start address constants): all registers in this list match, except the DSP commands. IIC.vi writes W_Debug_AWS = **63** (not 1) and W_Daul_AWS = **64** (not 5), and W_Daul_AWS / W_Mission_AWS each write a second byte, **CFh:80h = 63**, right after CFh:86h.

> Python now writes the same. The ShowPF check (CFh:86h = 0, CFh:81h = 0) was already right. Other IIC.vi details: the Temp Off Func commands use password 'AOIW' and all other commands 83 4A 2D 4C (Python: 'AOIW' to read, 83 4A 2D 4C to write; works on hardware).

> **Confirmed on hardware 2026-10-19** (commit 0da01aa, `regcheck` on P172262504894, finished by the LabVIEW V6.8 station): TempOffFunc, TestMode, DSP TestMode, DSP OperatingMode, AWS_Test_Mode, Bias_setpoint RT/HT/LT, W_IBiasPID and TOPPID (tag HW in `registers_aws.yaml`). Still decoded from IIC.vi only (tag CODE): DSP command CFh:80h, Vtops, tops_off and tops_room.

15. Units and formulas

Conversion	Formula	Example
dBm → linear (0.1 µW), calibration fits	10^(dBm/10) × 10000, **not rounded** (DDMI_Cal.vi: x/10, 10^x, ×10000)	−2 → 6309.57; −10 → 1000
DDMI power (R_DTXP / R_DRXP) → dBm	10·log10(raw × 0.0001 mW) = 10·log10(raw ÷ 10000) (IIC.vi)	11629 → 0.66 dBm
Volts → linear (100 µV)	V × 10000	3.13 → 31300
Temperature	S16 ÷ 256	2A80h → 42.5 °C
Vtops / tops_off	round(V ÷ 5.0 × 4095)	1.47 V → 1204
IBias	mA × 100	60 mA → 6000
Tops_setpoint, TXP_MPDOut	round(TXMON ÷ MPD_OUT × 1000)	1850 / 2940 → 629
Two-point fit	slope = Δy/Δx, offset = y_1 − slope·x_1	module reports slope·raw + offset
True TX power	meter dBm + Tx Loss[ch]	
RX table	att = att_lo + (target − p_lo) ÷ (p_hi − p_lo) × (att_hi − att_lo)	CH1: −6 dBm → 7.74 dB
Target level (MPD scan)	(1 − x) × peak TXMON, x in [0.65, 0.56, 0.50, 0.44]	0.50 → 3 dB below peak
DDMI VCC (R_DVCC)	raw × 0.0001 V (IIC.vi)	33000 → 3.3 V
Calibration storage (AWS)	float32 bit pattern (IIC.vi type cast), TX: ECh lanes 1–4 / EDh lanes 5–8, offsets C0h + 4·i, slopes D0h + 4·i; RX: offsets 90h + 4·i, slopes A0h + 4·i; VCC: EBh offset 90h, slope 94h	

> **Checked 2026-10-20** against the conversions inside IIC.vi (AWS case) and DDMI_Cal.vi: temperature ÷ 256, VCC × 0.0001 V, IBias ÷/× 100, Vtops V ÷ 5.0 × 4095 (other customers 2.5), DDMI power 10·log10(raw × 0.0001), calibration float layout, RX table lookup (target with RxP Offset 0, negated, interpolated) and the two-point linear fit all match the Python code. The fit inputs are unrounded since the section 5 / 6 changes.

16. Open items and differences still to close

#	Item	Section	Next action
1	Third comparison in the MPD pairing AND gate **OPEN**; CH6 of P172262504894 uses a different edge than Python	2	LabVIEW correlation run on a fresh module (DDMI only), compare edge choice per lane
2	Tx Loss on this bench	5, 11	Measure with the golden sample on this bench's fibres
3	Production value of "ATT Delay(s)" (Python uses the 3 saved in the VI); V6.8 waits it + 2 s per RX pair	6, 11	Read the Set tab on the production station; change `rx_cal.att_delay_s` if it differs
4	Sweep step wait 0.3 s vs 50 ms	2	Kept: required by this station's TXMON refresh rate

5	W_IBiasPID value after the TX cal **CLOSED**: V6.8 writes 15 (the 16430 was the first 2 bytes of the DBL 15.0)	5	–